

Chapter 04: The Limits of Linear Models

Table of Contents

[Chapter Introduction](#)

[Linear Models](#)

[What is Linear?](#)

[Linearly Separable Dataset](#)

[Linearly Inseparable Dataset](#)

[Why Perceptron fails on XOR or Linearly Inseparable Dataset](#)

[Historical Impact of Perceptron's Failure on XOR](#)

[The Brilliant Idea](#)

[Universal Approximation Theorem](#)

[An Intuition](#)

[Chapter Epilogue](#)

[In short](#)

Chapter Introduction

Our Perceptron Neuron learns. It classifies. It improves.

So, Why wasn't the Perceptron enough?

Because researchers discovered a deceptively simple dataset that defeated it completely. No matter how long the Perceptron trained, it **could never learn the correct answer**.

That dataset was the **XOR Dataset**.

Linear Models

In machine learning and statistics, a model is defined as "linear" if the decision boundary is linear in the feature space, or if the parameters are combined linearly.

Linear Regression, Logistic Regression, Perceptron and Linear SVM are examples of Linear Models.

$$\begin{aligned}
 \text{Weighted Sum, } z &\equiv z_{\vec{W}, b}(\vec{X}) \\
 &\equiv \vec{W} \cdot \vec{X} + b \\
 &\equiv w_1x_1 + w_2x_2 + w_3x_3 + \dots + w_ix_i + \dots + w_nx_n + b \\
 &\equiv \left(\sum_{i=1}^{i=n} w_ix_i \right) + b
 \end{aligned}$$

Each of Linear Regression, Logistic Regression, Perceptron and Linear SVM computes this Weighted Sum z . They differ in what they do with this z .

What is Linear?

In 2D Feature Space, a linear decision boundary ($w_1x_1 + w_2x_2 + b = 0$) is a straight line.

In 3D Feature Space, a linear decision boundary ($w_1x_1 + w_2x_2 + w_3x_3 + b = 0$) is a straight line.

Generally, in n -D Feature Space, a linear decision boundary ($w_1x_1 + w_2x_2 + w_3x_3 + \dots + w_ix_i + \dots + w_nx_n + b = 0$) is called a hyperplane.

The key idea is the same: one flat surface divides the space.

Linearly Separable Dataset

A dataset is linearly separable if a single linear hyperplane can perfectly separate its classes in its feature space.

The Perceptron can solve the Binary Classification Problem, on a Linearly Separable Binary Classification Dataset.

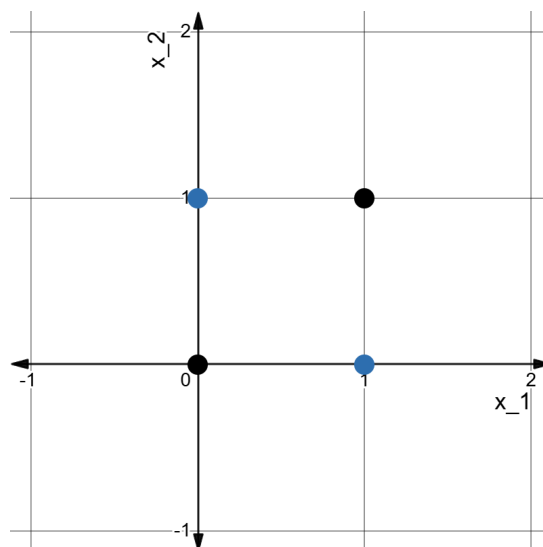
Linearly Inseparable Dataset

The Perceptron will continue updating its weights forever (or until a stopping criterion is reached), but it will never find a perfect separator of classes in the feature space of a Linearly Inseparable Dataset.

The (2-Feature) XOR Dataset below is such an Linearly Inseparable Dataset.

x_1	x_2	Target
0	0	0
0	1	1
1	0	1
1	1	0

The feature space (below) of this dataset has only 4 data points. Black points are of class 0, while blue points are of class 1.



If we try to separate these classes using a single straight line (that is, a 2D Hyperplane), we won't succeed. Every possible line misclassifies at least one point. This is a hallmark of the XOR dataset: it is **linearly inseparable**.

Why Perceptron fails on XOR or Linearly Inseparable Dataset

The Perceptron is limited to learning **one** linear decision boundary. Since XOR requires a nonlinear boundary, the Perceptron has no solution to discover. This is not a flaw in the training algorithm—it is a limitation of the model itself. No amount of additional training can overcome a model that lacks the necessary expressive power.

Historical Impact of Perceptron's Failure on XOR

In the late 1960s, researchers Marvin Minsky and Seymour Papert analyzed the Perceptron and showed that it could not solve problems like XOR with a single layer.

This observation was mathematically correct.

Unfortunately, many people interpreted it as meaning that **neural networks in general were fundamentally limited**.

Research funding declined, enthusiasm faded, and the field entered what is now known as the **AI Winter**.

Ironically, the real lesson was not that neural networks were impossible—it was that **single-layer** neural networks were insufficient.

The Brilliant Idea

Suppose one neuron cannot solve XOR.

What if we use two? Or ten? Or one hundred?

Instead of one decision boundary, we could combine several simple boundaries to create a much more complex one. Yes, the inspiration from Biological Brain: **Intelligence may emerge from many simple processing units working together.**

This was the birth of the **Multi-Layer Perceptron (MLP)**.

Many simple linear transformations, combined with nonlinear activation functions, can represent highly complex relationships.

Universal Approximation Theorem

Researchers later proved a remarkable result known as the **Universal Approximation Theorem**.

Simply put,

A neural network with at least one hidden layer and suitable nonlinear activation functions can approximate a very wide class of continuous functions, given enough neurons.

Notice the wording carefully:

- It says “can approximate”, not “will automatically learn”.
- It requires nonlinear activation functions.
- It does not imply that one hidden layer is always the most practical architecture.

We'll revisit this theorem after we've built multi-layer networks.

An Intuition

The image below is from [No Time for Flashcards](#).



Imagine trying to cut a complicated shape out of paper.

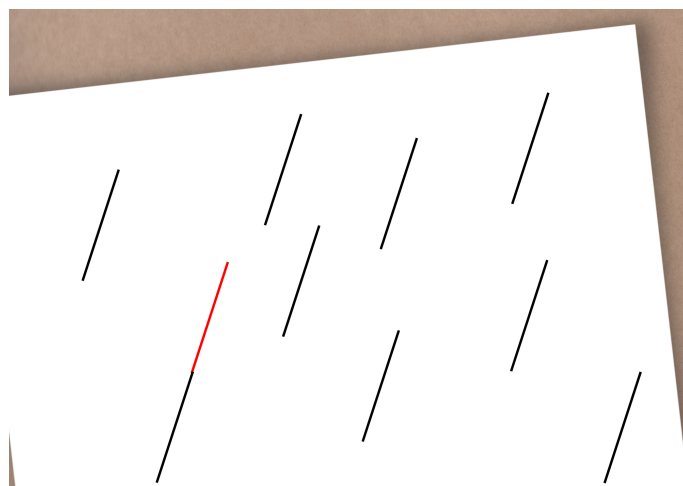
One straight cut cannot produce the shape.

But **many straight cuts, made in sequence, can approximate almost any outline.**

A single neuron is like one straight cut.

A deep neural network with non-linear activation functions is like thousands or millions of coordinated cuts, each cutting line intersecting the previous one, but each turning to left or right of the previous cutting line, as required by the shape, each refining the representation learned by the previous layers.

Well, then what about **a deep neural network with linear activation functions**? It's like thousands or millions of non-coordinated cuts, each cutting line not intersecting the previous one, each cutting line being either parallel or identical to one another, each not turning to left or right starting the previous cutting line,, each either not refining or not building upon the representation learned by the previous layers, as in the picture below.



Each line segment on the paper represents a scissor cut. Imagine, each scissor cut is done by a Neuron from the neural network.

With Linear Activation Function, the cuts are non-coordinated. The cuts are, either discrete from each other and parallel to each other, or on the same line (like the black and red pair of cuts). In

either or both types of cases combined, we don't get any shape out of the paper, at least no non-linear complex shape. That's why Deep Neural Networks use Non-Linear Activation Functions to approximate complex relationships.

Chapter Epilogue

Researchers could have accepted that artificial neurons were only useful for simple linear classifiers and moved on to other AI paradigms (symbolic AI, expert systems, etc.). Many did.

A smaller group instead pursued a new architecture: **stacking multiple layers** of neurons, inserting one or more “hidden” layers between the inputs and the final output. The resulting network is called a **Multi-Layer Perceptron (MLP)**.

With non-linear activation functions between layers, the network can compose multiple linear transformations into highly non-linear decision boundaries. Even a single hidden layer is, in theory, a universal approximator (it can represent any continuous function to arbitrary accuracy given enough depth and units). Multiple hidden layers enable hierarchical feature learning—exactly the principle that later became “deep” learning.

Inventing (or rather, seriously pursuing and eventually training) the MLP architecture turned neural networks from a limited historical curiosity into the scalable foundation of modern AI. The missing practical piece for many years was an efficient way to train the hidden layers (the credit-assignment problem). That was solved by the popularization of **backpropagation** in the mid-1980s (Rumelhart, Hinton, Williams and earlier precursors). Once training worked, deeper and wider networks became feasible, and the rest is the deep-learning revolution.

In short

Single neurons were too weak in expressive power.

→ Some researchers refused to stop there, while many did stop and move.

→ Researchers who continued pursuing found that multi-layer networks (MLPs) provided the necessary expressive power.

→ With proper training algorithms, they became the backbone of virtually all modern neural networks.